

10. `await` sirf Promise ke liye?

Practically `await` kisi bhi value ke saath use ho sakta hai:

```
</> JavaScript  
  
const x = await 10;  
console.log(x);
```

Result:

```
10
```

Lekin iska real benefit **Promises** ke saath hota hai.

11. Sequential vs Parallel — important

Ye:

```
</> JavaScript  
  
const user = await getUser();  
const products = await getProducts();
```

sequential hai.

```
getUser  
  ↓ complete  
getProducts  
  ↓ complete
```

Agar dono independent hain, to parallel kar sakte ho:

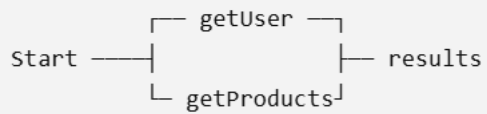
Api_part_1

Agar dono independent hain, to parallel kar sakte ho:

`</>` JavaScript

```
const [user, products] = await Promise.all([
  getUser(),
  getProducts()
]);
```

Flow:



Isse unnecessary waiting kam ho sakti hai.

12. Callback → Promise → async/await

Pura evolution yaad rakho:

```
Callback
  ↓
Callback Hell
  ↓
Promise
  ↓
Promise Chaining
  ↓
async/await
```

Callback

```
</> JavaScript

getUser(function(user) {
  console.log(user);
});
```

Promise

⌘ JavaScript

```
getUser()
  .then(user => console.log(user))
  .catch(err => console.log(err));
```

async/await

⌘ JavaScript

```
async function test() {

  try {
    const user = await getUser();
    console.log(user);
  } catch (err) {
    console.log(err);
  }

}
```

Keywords

Meaning

async

Function ko Promise-returning function banata hai

await

Promise settle hone tak current async function ko temporarily pause karta hai

try

Async code run karna

catch

Rejected Promise/error handle karna

Promise.all()

Multiple independent Promises ko parallel wait karna

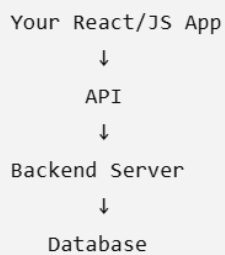
1. API kya hai?

API = Application Programming Interface

Simple language me:

API ek defined way hai jiske through ek software doosre software/service se data ya functionality maang sakta hai.

Example:



```
graph TD; A[Your React/JS App] --> B[API]; B --> C[Backend Server]; C --> D[Database];
```

Your React/JS App
↓
API
↓
Backend Server
↓
Database



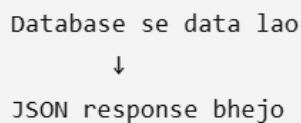
Frontend directly database se normally baat nahi karta.

Frontend directly database se normally baat nahi karta.

Frontend API ko request bhejta hai:

" Mujhe user id 10 ka data chahiye "

Backend:



```
graph TD; A[Database se data lao] --> B[JSON response bhejo];
```

Database se data lao
↓
JSON response bhejo

2. Real-life example

Restaurant ko samjho:

Customer → Waiter → Kitchen

Customer directly kitchen me jaakar order nahi banata.

Waiter communication ka medium hai.

Software me:

Frontend → API → Backend → Database

API ko waiter ki tarah samajh sakte ho.

API khud database nahi hai aur API khud backend bhi nahi hai.

3. JavaScript me API ka use

Browser JavaScript se server ko request bhejne ke liye commonly:

```
</> JavaScript
```

```
fetch()
```

use karte hain.

Example:

```
</> JavaScript
```

```
fetch("https://example.com/api/users")
```

Meaning:

"Server, mujhe users ka data do."

Meaning:

"Server, mujhe users ka data do."

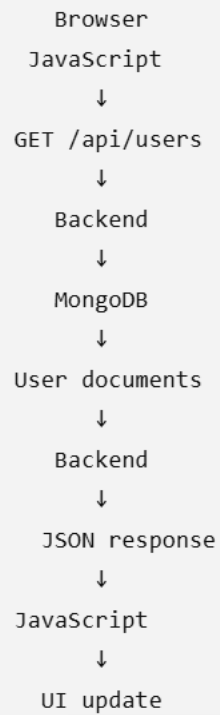
Server response deta hai:

`</>` JSON

```
[
  {
    "id": 1,
    "name": "Tarun"
  },
  {
    "id": 2,
    "name": "Rahul"
  }
]
```

4. API ka complete flow

Suppose tumhari website me **Users page** hai.



```
graph TD; Browser[Browser] --> JS1[JavaScript]; JS1 --> GET[GET /api/users]; GET --> Backend1[Backend]; Backend1 --> MongoDB[MongoDB]; MongoDB --> UserDoc[User documents]; UserDoc --> Backend2[Backend]; Backend2 --> JSON[JSON response]; JSON --> JS2[JavaScript]; JS2 --> UI[UI update];
```

The diagram illustrates the complete flow of an API request and response. It starts with a **Browser** sending a request to **JavaScript**, which then sends a **GET /api/users** request to the **Backend**. The **Backend** interacts with **MongoDB** to retrieve **User documents**. These documents are then processed by the **Backend** again to generate a **JSON response**. This response is received by **JavaScript**, which finally updates the **UI**.



Ye actual web-development me bahut common architecture hai.

5. API endpoint kya hota hai?

API ka specific address:

```
/api/users
```

endpoint ho sakta hai.

Example:

```
GET /api/users
GET /api/users/10
POST /api/users
PUT /api/users/10
DELETE /api/users/10
```

Inhe API endpoints/routes keh sakte hain.

7. JavaScript se GET API call

```
</> JavaScript

fetch("/api/users")
  .then(response => response.json())
  .then(data => {
    console.log(data);
  })
  .catch(error => {
    console.log(error);
  });
```

Modern way:

Modern way:

`</>` JavaScript

```
async function getUsers() {  
  
  try {  
  
    const response = await fetch("/api/users");  
  
    const data = await response.json();  
  
    console.log(data);  
  
  } catch (error) {  
  
    console.log(error);  
  
  }  
}
```



Yahan dekho tumhare previous concepts connect ho rahe hain:

Yahan dekho tumhare previous concepts connect ho rahe hain:

```
API  
↓  
fetch()  
↓  
Promise  
↓  
await  
↓  
JSON data
```

8. `fetch()` kya hai?

`fetch()` JavaScript/browser ka **Web API** hai.

Ye HTTP request bhejne ke liye use hota hai.

```
</> JavaScript
```

```
fetch(url)
```

`fetch()` **Promise** return karta hai.

Isliye:

```
</> JavaScript
```

```
const response = await fetch(url);
```

possible hai.

9. `response.json()` kya karta hai?

Server se response generally HTTP response ke form me aata hai.

```
</> JavaScript
```

```
const response = await fetch("/api/users");
```

`response` ke andar response information hoti hai.

JSON body ko JavaScript object/array me convert karne ke liye:

```
</> JavaScript
```

```
const data = await response.json();
```

Important:

```
fetch()
  ↓
Promise<Response>
  ↓
response.json()
  ↓
Promise<data>
  ↓
actual JavaScript object/array
```

10. POST API

Suppose new user create karna hai:

</> JavaScript

```
async function createUser() {

    const response = await fetch("/api/users", {

        method: "POST",

        headers: {
            "Content-Type": "application/json"
        },

        body: JSON.stringify({
            name: "Tarun",
            age: 22
        })

    });

    const data = await response.json();
```



Flow:

```
graph TD; A[JavaScript] --> B[POST request]; B --> C[/api/users/]; C --> D[Backend]; D --> E[Validate data]; E --> F[Database]; F --> G[Create user]; G --> H[JSON response]; H --> I[JavaScript];
```

The flowchart illustrates the sequence of operations for an API. It begins with JavaScript, which sends a POST request to the /api/users/ endpoint. This request is then processed by the Backend, which validates the data and interacts with the Database to create a new user. Finally, a JSON response is generated and sent back to the JavaScript client.

11. Request ke 4 important parts

API request ko generally is tarah samjho:

Request

├ URL

├ Method

├ Headers

└ Body

Example:

`</>` JavaScript

```
fetch("/api/users/10", {  
  method: "PATCH",  
  
  headers: {  
    "Content-Type": "application/json",  
    "Authorization": "Bearer TOKEN"  
  },  
  
  body: JSON.stringify({  
    name: "Tarun"
```



Api_part_1

```
    },  
  
    body: JSON.stringify({  
      name: "Tarun"  
    })  
  });
```

URL

```
/api/users/10
```

Kahan request bhejni hai.

Method

```
PATCH
```

Kya operation karna hai.

Headers

```
Authorization  
Content-Type
```

Extra information.

Body

```
</> JSON  
  
{  
  "name": "Tarun"  
}
```

Server ko bhejne wala data.

Api_part_1

200	Success
201	Successfully created
400	Bad request
401	Authentication required/invalid
403	Forbidden
404	Resource not found
500	Server error

Example:

`</>` JavaScript

```
if (!response.ok) {  
  throw new Error("API request failed");  
}
```



14. API + Express

Tum Node/Express padh rahe ho, to ye connection bahut important hai.

Backend:

`</>` JavaScript

```
const express = require("express");

const app = express();

app.get("/api/users", (req, res) => {

  res.json([
    {
      id: 1,
      name: "Tarun"
    }
  ]);

});

app.listen(5000);
```



Api_part_1

Frontend:

`</>` JavaScript

```
const response = await fetch("http://localhost:5000/api/users");

const users = await response.json();

console.log(users);
```

Flow:

```
React/HTML/JS
  ↓
fetch()
  ↓
Express API
  ↓
res.json()
  ↓
JavaScript
```



Third-party API kya hoti hai?

Jab API kisi doosri company/service ne banayi ho aur tum apni JavaScript application me usko use karo, to woh **Third-party API** hai.

Example:

```
Tumhari JS App
  ↓
Third-party API
  ↓
Google / Weather / Payment / Maps etc.
```



Example: Weather API

Tumhari website ko weather data chahiye.

Tum khud weather database maintain nahi karoge.

Third-party service ki API call karoge:

JavaScript

```
fetch("https://api.example.com/weather?city=Indore")
```

Response:

JSON

```
{
  "city": "Indore",
  "temperature": 28,
  "humidity": 70
}
```

Tum JavaScript me use karoge:



Tum JavaScript me use karoge:

JavaScript

```
async function getWeather() {
  const response = await fetch(
    "https://api.example.com/weather?city=Indore"
  );

  const data = await response.json();

  console.log(data.temperature);
}
```

API Key kya hoti hai?

Bahut si third-party APIs ke liye **API key** chahiye.

Example:

```
https://api.example.com/weather?city=Indore&appid=YOUR_API_KEY
```



API key basically service ko batati hai:

"Ye request kis registered application/user ki hai?"

Code me conceptually:

 JavaScript



```
const API_KEY = "abc123";

fetch(
  `https://api.example.com/weather?city=Indore&appid=${API_KEY}`
);
```

 **Secret API keys ko frontend JavaScript**  **expose nahi karna chahiye** agar provider key ko secret rakhta hai. Aise cases me:

1. API se kya-kya fetch kar sakte hain?

API kis type ka response deti hai, uske according:

Data	Example
Text	News, quotes, descriptions
JSON	Users, products, orders
Images	Product/profile images ke URLs
Videos	Video URLs/metadata
Audio	Music/audio URLs
Weather	Temperature, humidity
Maps/Location	Coordinates, places
Currency	Exchange rates
Movies	↓ Title, rating, poster

Api_part_1

```
async function getUsers() {  
  
  try {  
  
    const response = await fetch(  
      "https://example.com/api/users"  
    );  
  
    const data = await response.json();  
  
    console.log(data);  
  
  } catch (error) {  
  
    console.log(error);  
  
  }  
}  
  
getUsers();
```

3. API se images fetch/display karna

Suppose API response:

</> JSON

```
{
  "name": "Laptop",
  "price": 50000,
  "image": "https://example.com/laptop.jpg"
}
```

JavaScript:

</> JavaScript

```
async function getProduct() {

  const response = await fetch(
    "https://example.com/api/product"
  );

  const product = await response.json();
```



</> JavaScript

```
const product = await response.json();

console.log(product.name);
console.log(product.price);

document.querySelector("#productImage").src =
  product.image;
}
```

4. Multiple products + images

API:

</> JSON

```
[
  {
    "name": "Laptop",
    "price": 50000,
    "image": "laptop.jpg"
  },
  {
    "name": "Phone",
    "price": 25000,
    "image": "phone.jpg"
  }
]
```

Api_part_1

```
async function getProducts() {  
  
    const response = await fetch(  
        "https://example.com/api/products"  
    );  
  
    const products = await response.json();  
  
    products.forEach(product => {  
  
        console.log(product.name);  
        console.log(product.price);  
  
        const img = document.createElement("img");  
  
        img.src = product.image;  
  
        document.body.appendChild(img);  
  
    });  
}
```



5. Text data fetch

```
</> JavaScript

async function getQuote() {

    const response = await fetch(
        "https://example.com/api/quote"
    );

    const data = await response.json();

    document.body.innerText = data.quote;
}
```

6. Weather API

Conceptually:

```
</> JavaScript

async function getWeather() {

    const response = await fetch(
        "https://example.com/api/weather?city=Indore"
    );

    const data = await response.json();

    console.log(data.temperature);
    console.log(data.humidity);
    console.log(data.condition);
}
```

7. API se video information

API response:

JSON

```
{
  "title": "JavaScript Tutorial",
  "videoUrl": "https://example.com/video.mp4",
  "thumbnail": "https://example.com/thumb.jpg"
}
```

JavaScript:

JavaScript

```
async function getVideo() {

  const response = await fetch(
    "https://example.com/api/video"
  );

  const data = await response.json();

  document.querySelector("#video").src =
    data.videoUrl;

}
```

HTML:

HTML

```
<video id="video" controls></video>
```



8. API se POST data bhejna bhi possible hai

API sirf fetch/GET karne ke liye nahi hoti.

Tum data **send** bhi kar sakte ho.

</> JavaScript

```
async function createUser() {  
  
    const response = await fetch(  
        "https://example.com/api/users",  
        {  
            method: "POST",  
  
            headers: {  
                "Content-Type": "application/json"  
            },  
  
            body: JSON.stringify({  
                name: "Tarun",  
                age: 22  
            })  
        }  
    );
```



Api_part_1

```
const data = await response.json();  
  
console.log(data);  
}
```

Flow:

```
JavaScript  
  ↓  
POST API  
  ↓  
Backend  
  ↓  
Database  
  ↓  
Response  
  ↓  
JavaScript
```

9. Promise exactly kaha hai?

`fetch()` Promise return karta hai:

```
</> JavaScript

const promise = fetch(url);

console.log(promise);
```

Conceptually:

```
fetch()
  ↓
Promise
  ↓
Response
```

Isliye `.then()` use kar sakte ho:

```
</> JavaScript

fetch("/api/users")
  .then(response => response.json())
  .then(data => {
    console.log(data);
  })
  .catch(error => {
    console.log(error);
  });
```

10. Same thing async/await se

JavaScript

```
async function getData() {  
  
  try {  
  
    const response = await fetch("/api/users");  
  
    const data = await response.json();  
  
    console.log(data);  
  
  } catch (error) {  
  
    console.log(error);  
  
  }  
}
```

11. API se image ko `fetch()` karke Blob bhi bana sakte ho

Ye thoda advanced hai.

 JavaScript 

```
async function getImage() {  
  
    const response = await fetch(  
        "https://example.com/photo.jpg"  
    );  
  
    const blob = await response.blob();  
  
    const imageUrl = URL.createObjectURL(blob);  
  
    document.querySelector("img").src = imageUrl;  
}
```

Flow: